

Objetivos de la clase

- **Identificar** las diferencias entre errores y excepciones.
- **Utilizar** excepciones existentes.
- **Crear** excepciones propias.

Errores y excepciones

Aprendiendo del error

En la programación, se aprende a base de equivocarse, por lo tanto, debemos tolerar nuestros propios fallos para poder avanzar.

Programar no es fácil, podemos equivocarnos desde la concepción de la idea o también al escribir código, para lo primero poco podemos hacer, ya que es algo que mejorará con la práctica.

¡A programar se aprende programando y cometer errores es la prueba de que estás avanzando!

Errores y excepciones

Hasta ahora los mensajes de error no habían sido más que mencionados, pero si probaste los ejemplos probablemente hayas visto algunos. Hay (al menos) dos tipos diferentes de errores: errores de sintaxis y excepciones.

Cuando el programa falla ocasiona su detención, por lo cual tenemos que ser capaces de detectar por qué y cómo prevenir estas fallas.

Errores

Errores

Si algo tienen la mayoría de lenguajes de programación es que son exigentes con las instrucciones. Cuando en nuestro programa ocurre algún fallo Python nos lanzará un aviso indicando que ocurrió y detendrá la ejecución.

Los errores sintácticos están ligados a la sintaxis, que es la escritura de las instrucciones.



Errores sintácticos

Algunos de los errores sintácticos más comunes:

- ✓ Cerrar paréntesis
- ✓ Cerrar comillas o utilizar distintas comillas al abrir y cerrar
- ✓ Tabular mal el código
- ✓ No poner dos puntos al definir una función



Errores sintácticos



```
>>> print("Hola"
```

```
File "<ipython-input-1-8bc9f5174855>", line 1
```

```
    print("Hola"
```

```
        ^
```

```
SyntaxError: unexpected EOF while parsing
```

Vamos a provocar algunos errores

EOF significa **end of file**, el error nos indica que esperaba que se cierre con un paréntesis.

Errores de nombre



Se producen cuando el sistema interpreta que debe ejecutar alguna función, pero no está definida. Devuelve `NameError`:

```
>>> pint("Hola")
<ipython-input-2-155163d628c2> in <module>()
----> 1 pint("Hola")
```

```
NameError: name 'pint' is not defined
```

Pint no es la función print para
imprimir por pantalla.

Errores semánticos

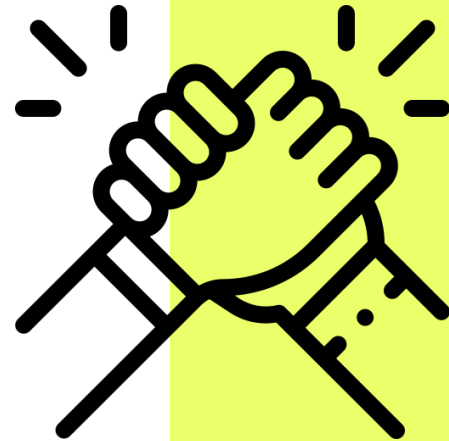
La mayoría de errores sintácticos y de nombre los identifican los editores de código antes de la ejecución, pero existen otros tipos que pasan más desapercibidos.

Estos errores son muy difíciles de identificar porque van ligados al sentido del funcionamiento y dependen de la situación. Algunas veces pueden ocurrir y otras no.



Errores semánticos

La mejor forma de prevenirlos es programando mucho y aprendiendo de tus propios fallos, **la experiencia es la clave**. Veamos a continuación algunos ejemplos.



Errores semánticos



```
>>> lista = []  
>>> lista.pop()  
<ipython-input-6-9e6f3717293a> in <module>()  
----> 1 l.pop()
```

IndexError: pop from empty list

Si intentamos sacar un elemento de una lista vacía, algo que no tiene mucho sentido, el programa dará fallo de tipo `IndexError`.

Esta situación ocurre sólo durante la ejecución del programa, por lo que los editores no lo detectarán:

Errores semánticos



```
>>> lista = []  
>>> if len(lista) > 0:  
    lista.pop()
```

Para prevenir el error deberíamos comprobar que una lista tenga como mínimo un elemento antes de intentar sacarlo, algo factible utilizando la función `len()`:

Errores semánticos



Cuando leemos un valor con la función `input()`, éste siempre se obtendrá como una cadena de caracteres. Si intentamos operarlo directamente con otros números tendremos un fallo `TypeError` que tampoco detectan los editores de código:

```
▶ n = input("Ingresar n:\n")  
  
m = 4  
  
print(f"---> {n}/{m} = {n/m}")
```

Errores semánticos



```
>>> n = float(input("Introduce un número: "))  
>>> m = 4  
>>> print(f"{n}/{m} = {n/m}")
```

Como ya sabemos este error se puede prevenir transformando la cadena a entero o flotante:



Desafío de errores

Retorna None cuando sucede el fallo

Duración: 10 minutos



ACTIVIDAD EN CLASE

Desafío de errores

Descripción de la actividad.

En la función de nuestro ejercicio hay un fallo potencial, averigua cuándo sucede y retorna el valor None en ese caso especial, en cualquier otro caso devuelve el resultado normal de la función.

```
>>> def dividir(a, b):  
    return a/b
```

Pista: Valor indeterminado

Break

¡5 minutos y volvemos!

Excepciones

Excepciones

Incluso si la sentencia o expresión es sintácticamente correcta, puede generar un error cuando se intenta ejecutarla. Los errores detectados durante la ejecución se llaman excepciones, y no son incondicionalmente fatales: pronto aprenderás cómo manejarlos en los programas en Python.



Excepciones



```
>>> 10 * (1/0)
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in ?
```

```
ZeroDivisionError: integer division or modulo by  
zero
```

```
>>> 4 + spam*3
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in ?
```

```
NameError: name 'spam' is not defined
```

Sin embargo, la mayoría de las excepciones no son manejadas por los programas, y resultan en mensajes de error como los mostrados aquí:

Excepciones

Como podemos suponer, es difícil prevenir fallos que ni siquiera nos habíamos planteado que podían existir. Por suerte para esas situaciones existen las excepciones.

Las excepciones son bloques de código que nos permiten continuar con la ejecución de un programa pese a que ocurra un error.

Try- Except

Try-Except



```
try:
    n = float(input("Ingresar n:\n")) #input dan una cadena
    m = 4
    print(f"---> {n}/{m} = {n/m}")
except:
    print("Algo salio!!!!: (.")
```

Para prevenir el fallo debemos poner el código propenso a errores en un **bloque try** y luego encadenar un **bloque except** para tratar la situación excepcional mostrando que ha ocurrido un fallo:

Try- Except

La sentencia **try** funciona de la siguiente manera:

- ✓ Se ejecuta el bloque try (el código entre las sentencias try y except).
- ✓ Si no ocurre ninguna excepción, el bloque except se saltea y termina la ejecución de la sentencia try.



Try- Except

La sentencia try funciona de la siguiente manera 👉

- ✓ Si ocurre una excepción durante la ejecución del bloque try, el resto del bloque se saltea. Luego, si su tipo coincide con la excepción nombrada luego de la palabra reservada except, se ejecuta el bloque except, y la ejecución continúa luego de la sentencia try.



Try- Except

- ✓ Si ocurre una excepción que no coincide con la excepción nombrada en el except, esta se pasa a declaraciones try de más afuera; si no se encuentra nada que la maneje, es una excepción no manejada, y la ejecución se frena con un mensaje como los mostrados arriba.



Try-Except

Try – Except **nos permite controlar situaciones excepcionales que generalmente darían error** y en su lugar permite mostrar un mensaje o ejecutar una pieza de código alternativo.

Else

Else

Las declaraciones try - except tienen un bloque else opcional, el cual, cuando está presente, debe seguir a los except. El bloque else es un buen momento para romper la iteración con break si todo funciona correctamente.

El uso de else es mejor que agregar código adicional en el try porque evita capturar accidentalmente una excepción que no fue generada por el código que está protegido por la sentencia try - except.

Else en ejemplo



```
while (True):
    try:
        a = float(input("Introduce un número: "))
        b = float(input("Introduce otro número: "))
        print(a + b)
    except:
        print("Ha ocurrido un error. Tienes que introducir 2 números.")
    else:
        print("La suma se ha realizado correctamente.")
        break # Importante romper la iteración si todo ha ido bien.
```

Finally

Finally

La sentencia try tiene otra sentencia opcional que intenta definir acciones de limpieza que deben ser ejecutadas bajo ciertas circunstancias. Una sentencia finally **siempre es ejecutada antes de salir de la sentencia try**, ya sea que una excepción haya ocurrido o no.

Finally

Cuando ocurre una excepción en la sentencia `try` y no fue manejada por una sentencia `except` (u ocurrió en una sentencia `except` o `else`), es lanzada luego de que se ejecuta la sentencia `finally`.

La sentencia `finally` es también ejecutada «a la salida» cuando cualquier otra sentencia de la sentencia `try` es dejada vía `break`, `continue` or `return`.

Finally en ejemplo



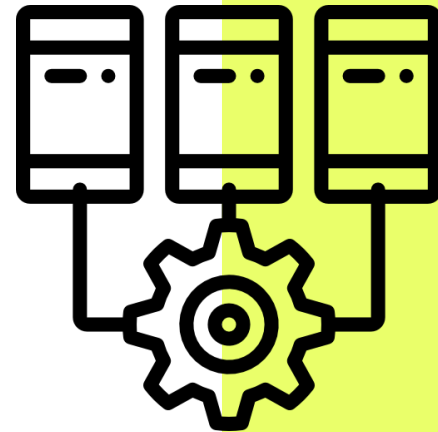
```
while (True):  
    try:  
        a = float(input("Introduce un número: "))  
        b = float(input("Introduce otro número: "))  
        print(a + b)  
    except:  
        print("Ha ocurrido un error. Tienes que introducir 2 números.")  
    else:  
        print("La suma se ha realizado correctamente.")  
        break # Importante romper la iteración si todo ha ido bien.  
    finally:  
        print("Fin del bucle") # Esto se ejecuta siempre.
```

Excepciones múltiples

Excepciones múltiples

En una misma pieza de código pueden ocurrir muchos errores distintos y quizá nos interese actuar de forma diferente en cada caso.

Para esas situaciones algo que podemos hacer es asignar una excepción a una variable.



Excepciones múltiples



De esta forma es posible analizar el tipo de error que sucede gracias a su identificador:

```
>>> try:
    n = input("Introduce un número:") # no transformamos
    a número
        5/n
except Exception as e: # guardamos la excepción como una
variable e
    print("Ha ocurrido un error =>",
type(e).__name__)
```

Excepciones múltiples



```
>>> print( type(e) )
```

Cada error tiene un identificador único que curiosamente se corresponde con su tipo de dato. Aprovechando eso podemos mostrar la clase del error utilizando la sintaxis:

Excepciones múltiples



```
>>> print(type(1))
```

```
>>> print(type(3.14))
```

```
>>> print(type([]))
```

```
>>> print(type(()))
```

Es similar a conseguir el tipo (o clase) de cualquier otra variable o valor literal:

Excepciones múltiples



```
>>> print( type(e).__name__ )
>>> print(type(1).__name__ )
>>> print(type(3.14).__name__ )
>>> print(type([]).__name__ )
>>> print(type(()).__name__ )
```

Como vemos, siempre nos indica "class" delante. Eso es porque en Python todo son clases, pero hablaremos de este concepto más adelante. Lo importante ahora es que podemos mostrar solo el nombre del tipo de dato (la clase) consultando su propiedad especial `name` de la siguiente forma:

Excepciones múltiples

Gracias a los identificadores de errores podemos crear múltiples comprobaciones, siempre que dejemos en último lugar la excepción por defecto. Excepción que engloba cualquier tipo de error.

Veamos a continuación un ejemplo

Excepciones múltiples



```
>>> try:
    n = float(input("Introduce un número divisor: "))
    5/n
except TypeError:
    print("No se puede dividir el número entre una cadena")
except ValueError:
    print("Debes introducir una cadena que sea un número")
except ZeroDivisionError:
    print("No se puede dividir por cero, prueba otro número")
except Exception as e:
    print("Ha ocurrido un error no previsto", type(e).__name__ )
```



Desafío de excepciones

Captura la excepción

Duración: 5 minutos



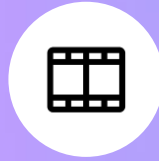
ACTIVIDAD EN CLASE

Descripción de la actividad.

Tiempo estimado: 5 minutos

Tomando la solución del ejercicio anterior, en lugar de devolver None al dividir entre cero, tienes que capturar una excepción que muestre por pantalla EXACTAMENTE el mensaje: “No se puede dividir entre cero”; si falla el código

```
>>> def dividir(a, b):  
    if b == 0:  
        return None  
    return a/b
```



¿Quieres saber más?
**Te dejamos material
ampliado de la clase**



MATERIAL AMPLIADO

Recursos multimedia

Título

- ✓ Artículo: [Errores](#)
- ✓ Artículo: [Excepciones](#)
- ✓ Artículo: [Else](#)
- ✓ Artículo: [Finally](#)
- ✓ Artículo: [Excepciones integradas](#)
- ✓ [Repaso Ejercicios](#)

¿Preguntas?

Resumen

de la clase hoy

- ✓ Errores
- ✓ Excepciones
- ✓ Excepciones Múltiples

Opina y valora
esta clase

Muchas gracias.

Programación Orientada a Objetos – I

Objetivos de la clase



Conocer qué es la POO.



Diferenciar la POO de la programación tradicional.



Aplicar el pensamiento computacional en un programa con objetos.

¿Qué es la POO?

¿Qué es la POO?

Es un modo o paradigma de programación, que nos permite organizar el código pensando el problema como una relación entre "cosas", denominadas objetos. Los objetos se trabajan utilizando las "clases". Estas nos permiten agrupar un conjunto de variables y funciones que veremos a continuación.

Motivación

Motivación

Los programadores se han dedicado a construir aplicaciones muy parecidas que resolvían una y otra vez los mismos problemas. Para conseguir que los esfuerzos de los programadores puedan ser reutilizados se creó la posibilidad de utilizar módulos. El primer módulo existente fue la función.

Pero la función se centra más en aportar una funcionalidad dada, pero no tiene tanto interés con los datos. 🙋

Motivación

Con la POO se busca resolver aplicaciones cada vez más complejas, sin que el código se vuelva un caos. Además, se pretende dar pautas para realizar las cosas de manera que otras personas puedan utilizarlas y adelantar su trabajo, de manera que consigamos que el código se pueda reutilizar.



En resumen...

Lo importante de la POO es poder separar los problemas generales en **suma de pequeños problemas aislados**, para poder modelar la solución y en un futuro que cualquiera pueda utilizar varios de estos módulos creados por este paradigma.

Diferencias con la programación tradicional

¿Qué es un paradigma de programación?

Un paradigma de programación es un estilo de desarrollo de programas. Es decir, un **modelo para resolver problemas computacionales**.

Los lenguajes de programación se encuadran en uno o varios paradigmas a la vez a partir del tipo de órdenes que permiten implementar, algo que tiene una relación directa con su sintaxis.

Paradigma de programación

- ✓ **Imperativo:** Los programas se componen de un conjunto de sentencias que cambian su estado. Son secuencias de comandos que ordenan acciones a la computadora.
- ✓ **Declarativo:** Opuesto al imperativo. Los programas describen los resultados esperados sin listar explícitamente los pasos a llevar a cabo para alcanzarlos.
- ✓ **Lógico:** El problema se modela con enunciados de lógica del primer orden.

Paradigma de programación

- ✓ **Funcional:** Los programas se componen de funciones, es decir, implementaciones de comportamiento que reciben un conjunto de datos de entrada y devuelven un valor de salida.
- ✓ **Orientado a Objetos:** El comportamiento del programa es llevado a cabo por objetos, entidades que representan elementos del problema a resolver y tienen atributos y comportamiento.

Diferencias con la programación tradicional

Programación estructurada

- ✓ Énfasis en la transformación de datos.
- ✓ Las funciones y los datos son manejados como entidades separadas.
- ✓ Difícil de entender y modificar.

Programación orientada a objetos

- ✓ Énfasis en la abstracción de datos.
- ✓ Las funciones y los datos son encapsulados en una entidad.
- ✓ Facilita su mantenimiento y su comprensión es orientada al mundo real.

Ventajas de la programación orientada a objetos

Ventajas



No se encapsulan los atributos, ni el acceso a los atributos desde las funciones, que también están encapsuladas dentro de la clase.



Ofrece la posibilidad de heredar de unas clases a otras para que puedan acceder a los miembros de la clase padre, con lo cual podemos solucionar de una manera más eficiente, el ampliar el comportamiento de nuestros objetos o clases.

Ventajas



Podemos hacernos visualmente una idea más clara de cómo se puede comportar la clase, además de tener en el mismo sitio tanto las funciones que hacen que podamos manejar la clase como los datos que vamos a manejar, etc.

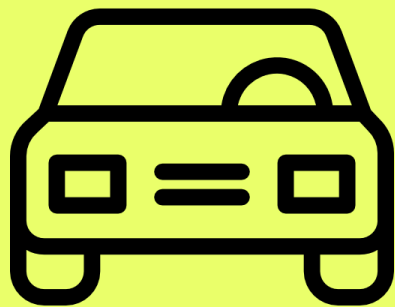
Break

**¿Cómo se piensa
con POO?**

¿Cómo se piensa con POO?

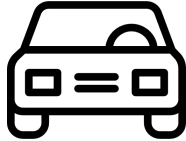
Es muy parecido a cómo lo haríamos en la vida real.
Por ejemplo: vamos a pensar en un coche para tratar de modelizar en un esquema de POO.

- ✓ **Elemento principal:** coche.
- ✓ **Características:** marca, modelo, color, etc.
- ✓ **Funcionalidades:** reversa, aparcamiento, etc.



¿Cómo se piensa con POO?

Ahora si pensamos el ejemplo anterior desde el esquema de Programación Orientada a Objetos, sería de la siguiente manera:



Elemento principal Clase	—————●
Características	—————●
	Atributos
Funcionalidades	—————●
	Métodos

¿Cómo se piensa con POO?

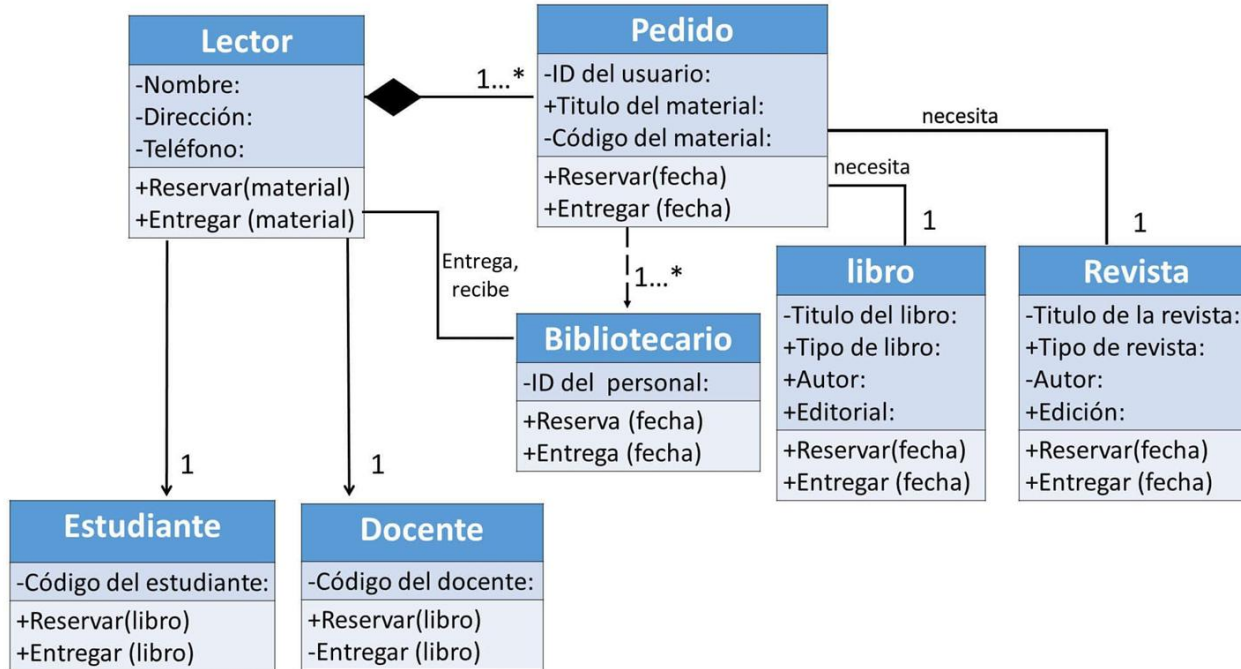
Para poder comentar el anterior esquema de POO fácilmente aparecen los denominados **diagramas de clases**.



Estructura estática que describe la estructura de un sistema mostrando las clases del sistema, sus atributos, operaciones, y las relaciones entre los objetos.

Ejemplos de diagramas de clases

Diagrama de clases de un sistema de servicios bibliotecarios



Con el ejemplo anterior notamos que los problemas complejos no son solo una suma de Clases, son además **relaciones entre clases.**

Relaciones entre clases

Tipos de relaciones

Tipos de relaciones

La Orientación a Objetos (POO) intenta modelar aplicaciones del mundo real tan fielmente como sea posible y, por lo tanto, debe reflejar estas relaciones entre clases y objetos.

Existen tres tipos de relaciones:

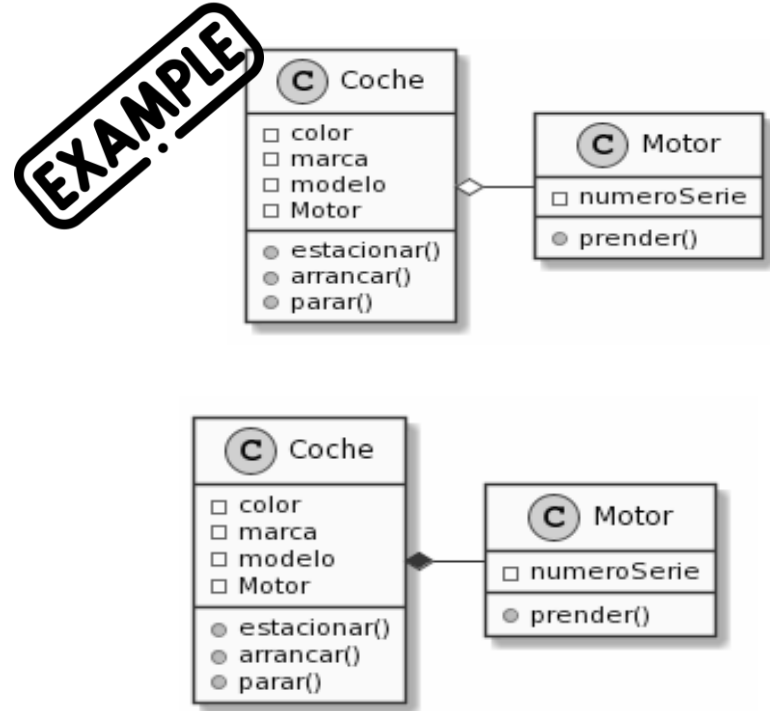
- ✓ Agregación / Composición
- ✓ Asociación
- ✓ Generalización / Especialización. Herencia simple y herencia múltiple.



1. Agregación

Esta relación se presenta entre una clase **TODO** y una clase **PARTE** que es **componente de TODO**.

La implementación de este tipo de relación se consigue definiendo como atributo un objeto de la otra clase que es parte-de. Los objetos de la clase **TODO** son objetos contenedores. **Un objeto contenedor es aquel que contiene otros objetos.**



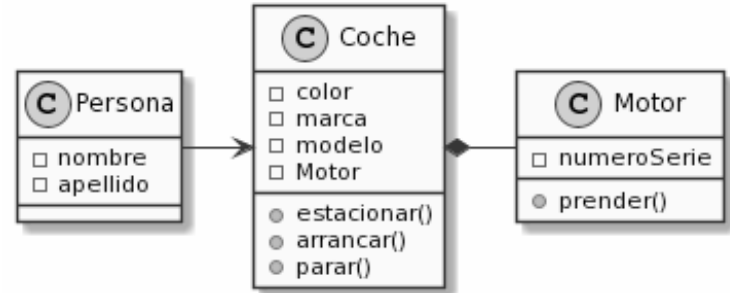
2. Asociación

Especifica una relación semántica entre objetos no relacionados. Este tipo de relaciones permiten crear asociaciones que capturan los participantes en una relación semántica.

Son relaciones del tipo **"pertenece_a"** o **"está_asociado_con"**.

Se da cuando una clase usa a otra clase para realizar algo.

EXAMPLE



3. Generalización

De todas las relaciones posibles entre las distintas clases y objetos, hay que destacar por su importancia en la POO el concepto de herencia. La relación de herencia es una relación entre clases que comparten su estructura y el comportamiento.

Resulta importante destacar que esta temática será abordada en próximos encuentros, por el momento únicamente definiremos los siguientes conceptos:

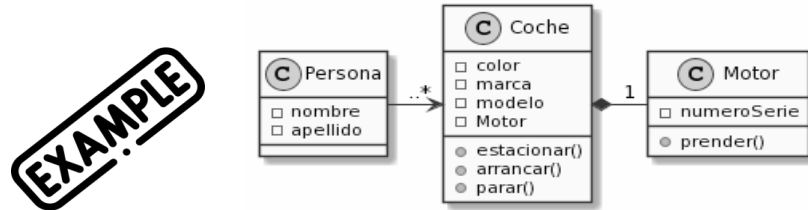
- ✓ **Herencia simple:** Una clase comparte la estructura y comportamiento de una sola clase.
- ✓ **Herencia múltiple:** Una clase comparte la estructura y comportamiento de varias clases.

Cardinalidad de las relaciones

Cardinalidad en las relaciones

Sabemos que un coche tiene un motor, y que la persona está asociada a un vehículo. Ahí nace el **concepto de cardinalidad**, es decir indicar el número de Objetos que están en la relación. Por ejemplo, una persona puede tener muchos coches, y que los coches solo tienen un motor.

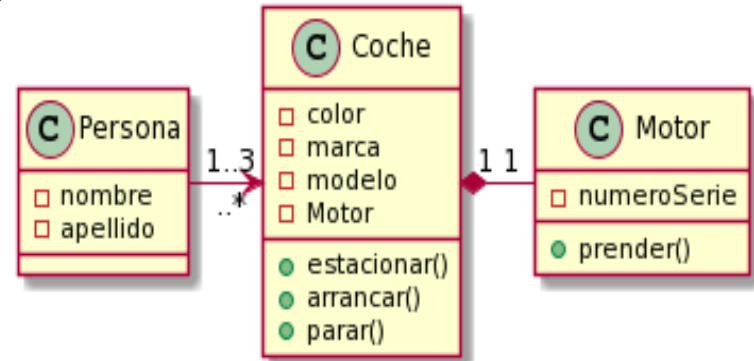
Además, sabemos que cada motor pertenece a un solo coche, y los coches a veces tienen más de un titular que lo usan, es decir tienen a varias personas para usarlo.



Cardinalidad en las relaciones

Solo por completitud del tema, y un poco alejado de la realidad, supongamos que un coche solo puede tener entre 1 y 3 titulares.

EXAMPLE



En la cardinalidad, lo importante es marcar si la relación es a muchos objetos ("**..***") o si solo es a uno (**1**), incluso a veces se puede decir a uno o ninguno (**0,1**).



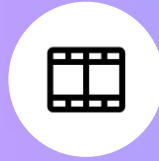
Ejemplo en vivo

¡Por último! Veamos un pequeño ejemplo de POO en Python.

Importante: El ejemplo que mostraremos a continuación es únicamente introductorio, en próximas sesiones se profundizará en la temática expuesta.

Notebook: *Clases_en_Python.ipynb*

¿Preguntas?



¿Quieres saber más?
**Te dejamos material
ampliado de la clase**



MATERIAL AMPLIADO

Recursos multimedia



[Ejemplo Clase](#)

Resumen de la clase hoy

- ✓ Programación Orientada a Objetos.
- ✓ Relaciones entre clases.
- ✓ Diagrama de clases.
- ✓ Diferencia entre POO y Tradicional.
- ✓ Ventajas de la POO.

Opina y valora
esta clase

Muchas gracias.

Clase 13. PYTHON

Programación orientada a Objetos II

Objetivos de la clase

- Conocer los métodos y atributos.
- Crear los primeros objetos y anidarlos.
- Identificar los principios del encapsulamiento.

¿Qué era la POO?

¿Qué era la POO?



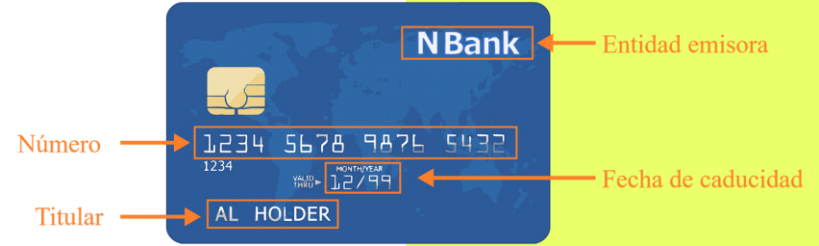
Es un modo o **paradigma de programación**, que nos permite **organizar el código pensando el problema como una relación entre "cosas"**, denominadas **objetos**.

Los objetos **se trabajan utilizando las clases**. Estas nos permiten **agrupar un conjunto de variables y funciones** que veremos a continuación. 👉

Atributos

Cosas de lo más cotidianas como una computadora o un coche pueden ser representadas con **clases**. Estas clases **tienen diferentes características**, que en el caso de la computadora podrían ser la marca o modelo. Llamaremos a estas características, **atributos**.

Atributos



Métodos

Activar



Pagar

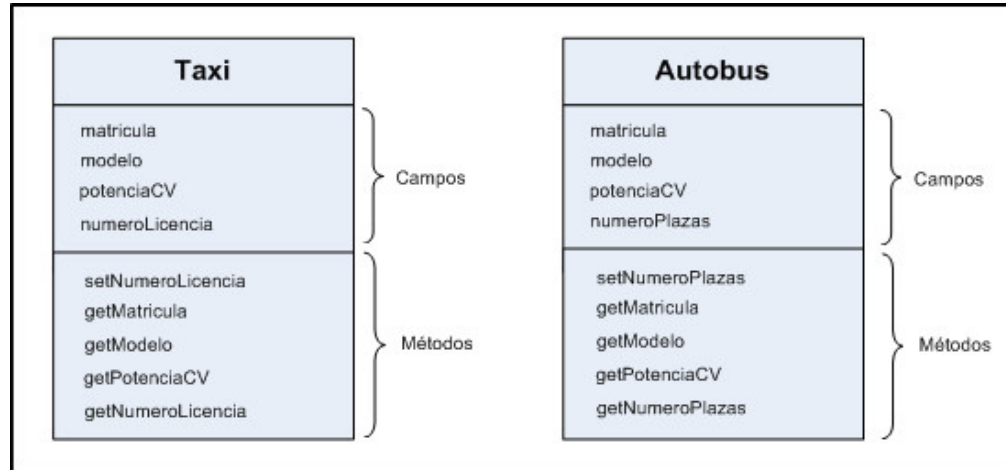


Anular



Métodos

Por otro lado, las **clases** tienen un conjunto de funcionalidades. En el caso de la computadora podría ser imprimir o reproducir. Llamaremos a estas funcionalidades **métodos**.



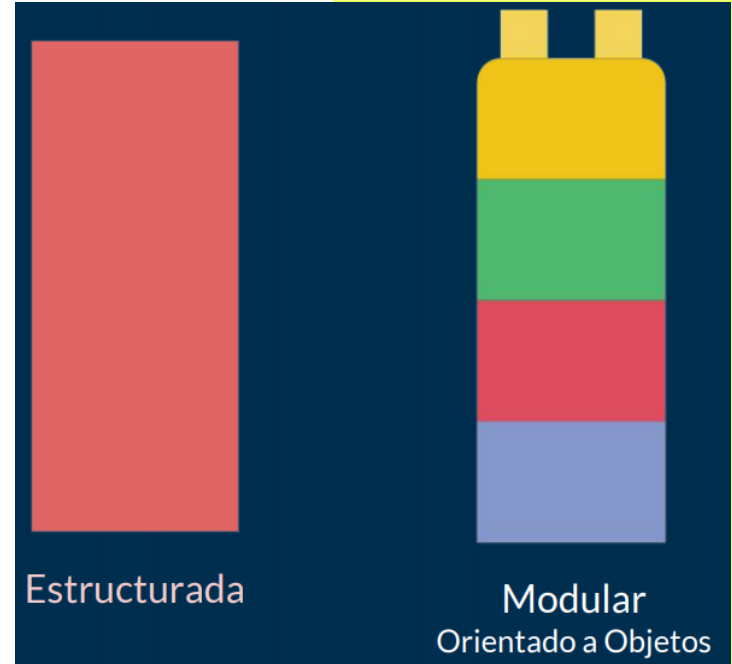
Objetos

Por último, pueden existir diferentes tipos de computadoras. Llamaremos a estos diferentes tipos de computadoras objetos.

Es decir, **el concepto abstracto de computadora es la clase, pero cualquier tipo de computadora particular será el objeto.**

¿Para qué usar la POO?

En el mundo de la programación hay gran cantidad de aplicaciones que realizan tareas muy similares y es importante identificar los patrones que nos permiten no reinventar la rueda. La programación orientada a objetos intentaba resolver esto.





PARA RECORDAR

Python está diseñado para soportar el paradigma de **la programación orientado a objetos (POO)**, donde **todos los módulos se tratan como entidades que tienen atributos, y comportamiento**. Lo cual, en muchos casos, ayuda a crear programas de una manera sencilla, además de que los componentes son reutilizables.

Clases

Uso de POO: clases



Lo primero es crear una clase, a la cual llamaremos “perro”. Se trata de una clase vacía y sin mucha utilidad práctica, pero es la mínima clase que podemos crear. El uso del PASS realmente no tiene utilidad, pero daría un error si después de los : no tenemos contenido.

```
1 class Perro:  
2     pass
```

Instanciar una clase



Ahora que tenemos la clase, podemos crear uno o más objetos de la misma, como si de una variable normal se tratase.

Nombre de la variable igual a la clase con (). Dentro de los paréntesis irían los parámetros de entrada si los hubiera.

```
class Perro:  
    pass  
  
perro1 = Perro()  
perro2 = Perro()
```

Atributos

Asignando atributos

A la hora de añadir atributos a la clase, es importante distinguir dos tipos:

- ✓ **Atributos de instancia:** Pertenecen a la instancia de la clase o al objeto. Son atributos particulares de cada instancia, en nuestro caso de cada perro.
- ✓ **Atributos de clase:** Se trata de atributos que pertenecen a la clase, por lo tanto serán comunes para todos los objetos.

Asignando atributos de instancia



Empecemos creando el nombre y la raza del perro. Esto se suele hacer por medio de un método `__init__` que será llamado automáticamente cuando se cree un objeto. No se le puede cambiar el nombre y tiene su definición propia, "**Constructor**".

```
class Perro:

    #Constructor de la clase
    def __init__(self, nombre, raza):

        #Atributos de la instancia
        self.nombre = nombre
        self.raza = raza
```



Para pensar

¿Para qué crees que se utiliza la variable SELF?

¿Verdadero o falso?

¿Qué es el self?

Es una variable que representa la instancia de la clase, y deberá estar siempre ahí.

El uso de `__init__` y el doble `__` no es una coincidencia. Cuando veas un método con esa forma, significa que está reservado para un uso especial del lenguaje. En este caso sería lo que se conoce como **constructor**.

Asignando atributos de clase



Hasta ahora hemos definido atributos de instancia, ya que pertenecen a cada perro concreto. Ahora vamos a definir un atributo de clase, que será común para todos los perros. Por ejemplo, la especie de los perros es algo común para todos los objetos Perro.

```
class Perro:

    #Atributos de clase
    especie = "Mamífero"

    #Constructor de la clase
    def __init__(self, nombre, raza):

        #Atributos de la instancia
        self.nombre = nombre
        self.raza = raza
```

Asignando atributos de clase



Dado que es un atributo de clase, no es necesario crear un objeto para acceder al atributo.

Se puede acceder también al atributo de clase desde el objeto.

👑 De esta manera, todos los objetos que se creen de la clase **perro** compartirán ese atributo de clase.

```
print(Perro.especie)  
# mamífero
```

```
mi_perro = Perro("Toby", "Bulldog")  
mi_perro.especie  
# 'mamífero'
```



Estructura final de una clase con atributos

```
14 class Perro:
15
16     #Atributos de clase
17     especie = "Mamífero"
18
19     #Constructor de la clase
20     def __init__(self, nombre, raza):
21
22         #Atributos de la instancia
23         self.nombre = nombre
24         self.raza = raza
25
26
27 perro1 = Perro("Sammy", "Caniche")
28
29 print(f"Su nombre es: {perro1.nombre}")
30 print(f"Su raza es: {perro1.raza}")
31 print(f"Es un: {perro1.especie}")
```

PROBLEMS OUTPUT **TERMINAL** DEBUG CONSOLE

Su nombre es: Sammy
Su raza es: Caniche
Es un: Mamífero

Métodos

Definiendo métodos

En realidad cuando usamos `__init__` ya estábamos definiendo un método especial.



A continuación, vamos a ver cómo definir métodos que le den alguna funcionalidad interesante a nuestra clase, siguiendo con el ejemplo de perro. 🐾

Definiendo métodos



Vamos a codificar dos métodos: **Ladrar** y **caminar**.

El primero no recibirá ningún parámetro y el segundo recibirá el número de pasos que queremos andar. Como hemos indicado anteriormente, **self** hace referencia a la instancia de la clase.

```
class Perro:
    # Atributo de clase
    especie = 'mamífero'

    # El método __init__ es llamado al crear el objeto
    def __init__(self, nombre, raza):
        print(f"Creando perro {nombre}, {raza}")

        # Atributos de instancia
        self.nombre = nombre
        self.raza = raza

    def ladra(self):
        print("Guau")

    def camina(self, pasos):
        print(f"Caminando {pasos} pasos")
```

Definiendo métodos



Se puede definir un método con **def** y el **nombre**, y entre **()** los **parámetros de entrada que recibe**, donde siempre tendrá que estar **self** el primero.

```
class Perro:
    # Atributo de clase
    especie = 'mamífero'

    # El método __init__ es llamado al crear el objeto
    def __init__(self, nombre, raza):
        print(f"Creando perro {nombre}, {raza}")

        # Atributos de instancia
        self.nombre = nombre
        self.raza = raza

    def ladra(self):
        print("Guau")

    def camina(self, pasos):
        print(f"Caminando {pasos} pasos")
```

Definiendo métodos



Por lo tanto, si creamos un objeto **mi_perro**, podremos hacer uso de sus métodos llamándolos con **.** y el nombre del método. Como si de una función se tratase, pueden recibir y devolver argumentos.



```
mi_perro = Perro("Toby", "Bulldog")
mi_perro.ladra()
mi_perro.camina(10)

# Creando perro Toby, Bulldog
# Guau
# Caminando 10 pasos
```

Nuestro código quedaría de la siguiente manera



```
1 class Perro:
2
3     #Atributos de clase
4     especie = "Mamífero"
5
6     #Constructor de la clase
7     def __init__(self, nombre, raza):
8
9         #Atributos de la instancia
10        self.nombre = nombre
11        self.raza = raza
12
13    #Métodos
14    def ladrar(self):
15        print("Este perro ha ladrado :( \nEstá enojado")
16
17
18    def caminar(self, pasos):
19        print(f"Este perro ha caminado {pasos} pasos")
20
21
22 perro1 = Perro("Sammy", "Caniche")
23
24 print(f"Su nombre es: {perro1.nombre}")
25 print(f"Su raza es: {perro1.raza}")
26 print(f"Es un: {perro1.especie}")
27
28 perro1.ladrar()
29 perro1.caminar(5)
30
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
Su raza es: Caniche
Es un: Mamífero
Este perro ha ladrado :(
Está enojado
Este perro ha caminado 5 pasos
```

Break

Métodos especiales

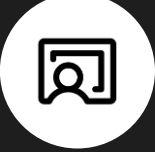
¿Qué son?

Los métodos mágicos de las clases Python son aquellos que comienzan y terminan con dos caracteres subrayados. Son de suma importancia ya que muchas de las operaciones que se hacen con clases en Python utilizan estos métodos, como puede ser la creación del objeto (`__init__`) o generar una cadena que los represente (`__str__`).

Métodos especiales

Todos estos métodos tienen un primer parámetro que hace referencia al propio objeto, que por convenio se suele llamar **self**. Siendo el resto de los parámetros del método variables.

Veamos a continuación algunos de estos métodos y cuál es su finalidad. 👉



```
1 class Perro:
2
3     #Atributos de clase
4     especie = "Mamífero"
5
6     #Constructor de la clase
7     def __init__(self, nombre, raza):
8
9         #Atributos de la instancia
10        self.nombre = nombre
11        self.raza = raza
12
13    #Métodos
14    def ladrar(self):
15        print("Este perro ha ladrado :( \nEstá enojado")
16
17
18    def caminar(self, pasos):
19        print(f"Este perro ha caminado {pasos} pasos")
20
21
22 perro1 = Perro("Sammy", "Caniche")
23
24 print(f"Su nombre es: {perro1.nombre}")
25 print(f"Su raza es: {perro1.raza}")
26 print(f"Es un: {perro1.especie}")
27
28 perro1.ladrar()
29 perro1.caminar(5)
30
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
Su raza es: Caniche
Es un: Mamífero
Este perro ha ladrado :(
Está enojado
Este perro ha caminado 5 pasos
```

`_init_`



El método mágico más utilizado en las clases de python es el constructor. Empleado para crear cada una de las instancias de las clases. No tiene una cantidad fija de parámetros, ya que estos dependen de las propiedades que tenga el objeto. Por ejemplo, se puede crear un objeto Vector:

```
class Vector():  
    def __init__(self, data):  
        self._data = data  
  
v = Vector([1,2])  
print(v)
```

str

Posiblemente el segundo método más utilizado, con el que se crea una representación del objeto con significado para las personas.

En el ejemplo anterior, **al imprimir el objeto, se ha visto una cadena que indica el tipo de objeto y una dirección**. Lo que generalmente no tiene sentido para las personas.

`__str__`



Un problema que se puede solucionar con el método `__str__`.

```
class Vector():  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values  
are: {self._data}"  
  
v = Vector([1,2])  
print(v)
```

`_len_`

Podríamos usar la función `len` para obtener el número de elementos que tenga el objeto.

Pero si probamos con el objeto nos encontraremos con un error. 🐞



```
class Vector():  
  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values are:  
{self._data}"  
  
    def __len__(self):  
        return len(self._data)  
  
v = Vector([1,2])  
len(v)
```

__len__

Esto es así porque la clase no ha implementado aún el método `__len__`. Un problema se puede solucionar de tomar fácil al incluir este método.



```
class Vector():  
  
    def __init__(self, data):  
        self._data = data  
  
    def __str__(self):  
        return f"The values are:  
{self._data}"  
  
    def __len__(self):  
        return len(self._data)  
  
v = Vector([1,2])  
len(v)
```

`__getitem__`

Si se desea que los usuarios puedan leer los elementos mediante el uso de corchetes es necesario implementar el método `__getitem__` en la clase. Este elemento necesita un parámetro adicional, que es la posición del elemento a leer. La función debe retornar el valor leído.



```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values
are: {self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self,
pos):
        return self._data[pos]

v = Vector([1,2])
v[1]
```



PARA RECORDAR

El método `__getitem__` solo nos sirve para leer, ya que para modificar, se tiene que crear también el método `__setitem__`, el cual veremos a continuación.



`__setitem__`

Este es el complemento del método anterior. En este caso es necesario pasar dos parámetros adicionales, la posición y el valor a reemplazar.

```
class Vector():
    def __init__(self, data):
        self._data = data

    def __str__(self):
        return f"The values are:
{self._data}"

    def __len__(self):
        return len(self._data)

    def __getitem__(self, pos):
        return self._data[pos]

    def __setitem__(self, pos, value):
        self._data[pos] = value

v = Vector([1,2])
v[1] = 20
print(v)
```

Clases anidadas

Relación entre clases

Cuando vimos el por qué trabajar por medio de este paradigma, dijimos que era fundamental para **separar a nuestro problema en problemas más pequeños (Clases)**, pero estos “problemas” no van a estar aislados, estarán **relacionados entre sí**. Por ejemplo, los **Perros**, suelen ser la mascota de una **Persona**.

Relación entre clases

Es muy simple. Pero en la próxima slide veremos otra variante de Clases Anidadas.



```
12
13 def __str__(self):
14     return "Nombre:" +self.nombre +", Raza:" +self.raza +", Especie:" +self.especie
15
16 #Métodos
17 def ladrar(self):
18     print("Este perro ha ladrado :( \nEstá enojado")
19
20
21 def caminar(self, pasos):
22     print(f"Este perro ha caminado {pasos} pasos")
23
24 class Persona:
25
26     def __init__(self, nombre, apellido, perro):
27         self.nombre = nombre
28         self.apellido = apellido
29         self.perro = perro
30
31
32     def __str__(self):
33         return "Mi nombre es: " +self.nombre + " " +self.apellido +"\n" +self.perro.__str__()
34
35 perro1 = Perro("Sammy", "Caniche")
36 persona1 = Persona("Nicolas", "Perez", perro1)
37
38 print(persona1)
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
TypeError: can only concatenate str (not "Perro") to str
PS C:\Users\nico\Desktop\Clase xx> & C:\Users\nico\AppData\Local\Microsoft\WindowsApps\python3.9.exe "c:/Users/nico/Desktop/Clase xx/example.py"
Mi nombre es: Nicolas Perez
Nombre:Sammy, Raza:Caniche, Especie:Mamifero
```

Relación entre clases



Esta es otra forma de realizar clases internas a otras clases. No es lo más típico y usado, pero puede ser útil para cuando hay una relación muy estricta entre dos clases.

```
41 class Sueldo:
42
43     def __init__(self, sueldo):
44         self.sueldo = sueldo
45
46     def __str__(self):
47         return f"\nSUELDO: {self.sueldo}"
48
49
50 class Empleado:
51
52     def __init__(self, nombre, puesto):
53         self.nombre = nombre
54         self.puesto = puesto
55         self.sueldo = Sueldo(1200)
56
57
58     def __str__(self):
59         return f"NOMBRE: {self.nombre}\nPUESTO: {self.puesto}\n" + self.sueldo.__str__()
60
61 s1 = Sueldo(200)
62 emp1 = Sueldo.Empleado("Nico", "Profe")
63
64 print("RESULTADO 1: " + s1.__str__())
65 print("RESULTADO 2: " + emp1.__str__())
```



Para pensar

¿Cómo explicarías los últimos print?

¿Qué se representa en el código de la siguiente slide?

```
41 class Sueldo:
42
43     def __init__(self, sueldo):
44         self.sueldo = sueldo
45
46     def __str__(self):
47         return f"\nSUELDO: {self.sueldo}"
48
49
50 class Empleado:
51
52     def __init__(self, nombre, puesto):
53         self.nombre = nombre
54         self.puesto = puesto
55         self.sueldo = Sueldo(1200)
56
57
58     def __str__(self):
59         return f"NOMBRE: {self.nombre}\nPUESTO: {self.puesto}\n" + self.sueldo.__str__()
60
61 s1 = Sueldo(200)
62 emp1 = Sueldo.Empleado("Nico", "Profe")
63
64 print("RESULTADO 1: " + s1.__str__())
65 print("RESULTADO 2: " + emp1.__str__())
```

```
RESULTADO 1:
SUELDO: 200
RESULTADO 2: NOMBRE: Nico
PUESTO: Profe
SUELDO: 1200
```

Encapsulamiento

¿De qué se trata?

Es un concepto relacionado con la programación, orientada a objetos, y hace referencia al **ocultamiento de los estados internos de una clase al exterior**. Dicho de otra manera, encapsular consiste en **hacer que los atributos o métodos internos a una clase no se puedan acceder ni modificar desde fuera**, sino que tan solo el propio objeto pueda acceder a ellos.



Python por defecto no oculta los atributos y métodos de una clase al exterior 😞



PARA RECORDAR

Esto es genial para “aprender”, pero en la práctica no nos gustaría que desde la clase Perro le cambien el nombre a una instancia de la clase Sueldo, por dar un ejemplo. Es por eso que es fundamental “defender” a los datos y tornarlos privados.



Para pensar

¿Qué creen que ocurre en este caso?

Pista

Atención a los _ _

```
class Persona:

    tipo = "Humano" #Dato al que pueden acceder
    __sueldo = 2000 #No quiero que accedan a mi cuenta

    def __init__(self, nombre, apellido):
        self.nombre = nombre #NO me molesta que sepan mi nombre
        self.__apellido = apellido #No quiero que sepan mi apellido

    def __soy_feliz(self):
        print("No les importa ~_~")

    def edad(self):
        return 31 #Soy joven aún no miento con mi edad

persona1 = Persona("Nicolas", "Lopez")

print(f"Resultado1: {persona1.tipo}\n")
print(f"Resultado2: {persona1.__sueldo}\n")
print(f"Resultado3: {persona1.nombre}\n")
print(f"Resultado4: {persona1.__apellido}\n")
print(f"Resultado5: {persona1.__soy_feliz()}\n")
print(f"Resultado6: {persona1.edad()}\n")
```



Para pensar

¿Qué creen que ocurre en este caso entonces?

```
85 |
86 persona1 = Persona("Nicolas", "Lopez")
87
88 print(f"Resultado1: {persona1.tipo}\n")
89 #print(f"Resultado2: {persona1.__sueldo}\n")
90 print(f"Resultado3: {persona1.nombre}\n")
91 #print(f"Resultado4: {persona1.__apellido}\n")
92 #print(f"Resultado5: {persona1.__soy_feliz()}\n")
93 print(f"Resultado6: {persona1.edad()}\n")
```

PROBLEMS

OUTPUT

TERMINAL

DEBUG CONSOLE

```
AttributeError: 'Persona' object has no attribute '__soy_feliz'
PS C:\Users\nico\Desktop\Clase xx> & C:/Users/nico/AppData/Local/Microsoft/
RESULTADO 1:
SUELDO: 200
RESULTADO 2: NOMBRE: Nico
PUESTO: Profe

SUELDO: 1200
Resultado1: Humano

Resultado3: Nicolas

Resultado6: 31
```

Contesta mediante el chat de Zoom

Visibilidad de los datos

Efectivamente, todo lo que esté privado (**con doble __**) será imposible de acceder desde el exterior de la clase





¿Cómo encapsular en Python?

Hasta el momento sabemos que **la encapsulación es el ocultamiento de datos del estado interno para proteger la integridad del objeto**. En el siguiente ejemplo, CLIENTE es una clase, y hemos encapsulado (al anteponer `__`) la variable `accountNumber`.

```
1 | class Customer:
2 |     def __init__(self):
3 |         self.__accountNumber = 4321
4 |     def getAccountNumber(self):
5 |         return self.__accountNumber
```



¿Cómo encapsular en Python?

Del mismo modo, podemos ocultar el acceso a los métodos. Para eso, necesitamos anteponer el nombre del método con `__` (subrayado doble).

```
1 class Customer:
2     def __init__(self):
3         self.__accountNumber = 4321
4
5     def __processAccount(self):
6         print("Processing Account")
7
8     def getAccountNumber(self):
9         return self.__accountNumber
```




Implementación en Python

Clases desde DrawIO a Python

Duración: **20 minutos**



ACTIVIDAD EN CLASE

Implementación en Python

Consigna: Implementar la Clase de Alumno, creada en la actividad anterior, directamente en Python.

Aclaraciones Generales:

- ✓ El método *imprimir*, deberá mostrar por pantalla el nombre y la nota del estudiante.
- ✓ El método *resultado*, evalúa la nota correspondiente del estudiante. Si el estudiante saca menos de 5 puntos desaprueba la materia, más de 5 puntos aprueba. **Tip:** Para la realización de este apartado, es necesario trabajar con estructuras condicionales.
- ✓ Se deberá instanciar, al menos, dos objetos pertenecientes a la clase **Alumno**.



¿Aún quieres conocer más?
Te recomendamos el
siguiente material



MATERIAL AMPLIADO

Recursos multimedia

Ejemplos de repaso



[EjemplodeRepaso](#)

¿Preguntas?

Clase 14. PYTHON

Herencias

Objetivos de la clase

- Aplicar Herencias.
- Conocer los polimorfismos.
- Ejecutar Herencias múltiples.

Herencia

¿Qué es?

La herencia es un proceso mediante el cual se puede crear una **clase hija** que hereda de una **clase padre**, compartiendo sus métodos y atributos.

Además de ello, una clase hija puede sobrescribir los métodos o atributos, o incluso definir unos nuevos.

¿De qué se trata?

```
# Definimos una clase padre
class Animal:
    pass

# Creamos una clase hija que hereda de la padre
class Perro(Animal):
    pass
```

Se puede crear una clase hija con tan solo pasar como parámetro la clase de la que queremos heredar.

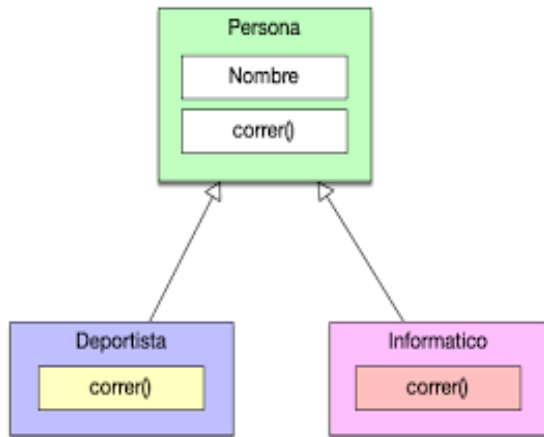
¿Cómo funciona?

Dado que una clase hija hereda los atributos y métodos de la padre, nos puede ser muy útil cuando tengamos clases que se parecen entre sí pero tienen ciertas particularidades.

En este caso, siguiendo el ejemplo anterior, en vez de definir muchas clases para cada animal, **podemos tomar los elementos comunes y crear una clase Animal de la que hereden el resto**, respetando por tanto la filosofía DRY.

Principio de DRY

Don't repeat yourself



Cuanto más código duplicado exista, más difícil será de modificar y más fácil será crear inconsistencias.

Las clases y la herencia ayudan a no repetir código de manera innecesaria.



PARA RECORDAR

Realizar **estas abstracciones y buscar el denominador común para definir una clase de la que hereden las demás**, es una tarea de lo más compleja en el mundo de la programación.



Ejemplo en vivo

Vamos a definir una clase padre **Animal** que tendrá todos los atributos y métodos genéricos que los animales pueden tener.



Atributos

```
class Animal:
    def __init__(self, especie, edad):
        self.especie = especie
        self.edad = edad

    # Método genérico pero con implementación particular
    def hablar(self):
        # Método vacío
        pass

    # Método genérico pero con implementación particular
    def moverse(self):
        # Método vacío
        pass

    # Método genérico con la misma implementación
    def describeme(self):
        print("Soy un Animal del tipo", type(self).__name__)
```

- ✓ **Especie:** todos los animales pertenecen a una.
- ✓ **Edad:** momento cronológico.



Métodos o funcionalidades

```
class Animal:
    def __init__(self, especie, edad):
        self.especie = especie
        self.edad = edad

    # Método genérico pero con implementación particular
    def hablar(self):
        # Método vacío
        pass

    # Método genérico pero con implementación particular
    def moverse(self):
        # Método vacío
        pass

    # Método genérico con la misma implementación
    def describeme(self):
        print("Soy un Animal del tipo", type(self).__name__)
```

- ✓ Método **hablar**: que cada animal implementará de una forma.
- ✓ Método **moverse**: Unos animales lo harán caminando, otros volando.
- ✓ Método **describeme** que será común a todos los animales.

Clase vacía

Tenemos ya por lo tanto una clase genérica **Animal**, que generaliza las características y funcionalidades que todo animal puede tener.

Ahora creamos una clase **Perro** que hereda del **Animal**. 🐾





Clase vacía

Como primer ejemplo vamos a crear una clase vacía, para ver cómo los métodos y atributos son heredados por defecto.

```
# Perro hereda de Animal
class Perro(Animal):
    pass

mi_perro = Perro('mamífero', 10)
mi_perro.describeme()
# Soy un Animal del tipo Perro
```



Creando clases

Con tan solo un par de líneas de código, hemos creado una clase nueva que tiene todo el contenido que la clase padre tiene.

A continuación, vamos a crear varios animales concretos y sobrescribir algunos de los métodos que habían sido definidos en la clase **Animal**, como el **hablar** o el **move**, ya que cada animal se comporta de una manera distinta.

```
class Perro(Animal):
    def hablar(self):
        print("Guau!")
    def move(self):
        print("Caminando con 4 patas")

class Vaca(Animal):
    def hablar(self):
        print("Muuu!")
    def move(self):
        print("Caminando con 4 patas")

class Abeja(Animal):
    def hablar(self):
        print("Bzzzz!")
    def move(self):
        print("Volando")

# Nuevo método
def picar(self):
    print("Picar!")
```

Creando clases

Podemos, incluso, crear nuevos métodos que se añadirán a los ya heredados, como en el caso de la **Abeja** con **picar()**.



```
class Perro(Animal):
    def hablar(self):
        print("Guau!")
    def moverse(self):
        print("Caminando con 4 patas")

class Vaca(Animal):
    def hablar(self):
        print("Muuu!")
    def moverse(self):
        print("Caminando con 4 patas")

class Abeja(Animal):
    def hablar(self):
        print("Bzzzz!")
    def moverse(self):
        print("Volando")

    # Nuevo método
    def picar(self):
        print("Picar!")
```





Creando clases

Por lo tanto ya podemos crear nuestros objetos de esos animales y hacer uso de sus métodos que podrían clasificarse en tres.



```
mi_perro = Perro('mamífero', 10)
mi_vaca = Vaca('mamífero', 23)
mi_abeja = Abeja('insecto', 1)

mi_perro.hablar()
mi_vaca.hablar()
# Guau!
# Muuu!

mi_vaca.describeme()
mi_abeja.describeme()
# Soy un Animal del tipo Vaca
# Soy un Animal del tipo Abeja

mi_abeja.picar()
# Picar!
```



Creando clases

- ✓ Heredados directamente de la clase padre: **describeme.**
- ✓ Heredados de la clase padre pero modificados: **hablar y moverse.**
- ✓ Creados en la clase hija por lo tanto no existentes en la clase padre: **picar.**

```
mi_perro = Perro('mamífero', 10)
mi_vaca = Vaca('mamífero', 23)
mi_abeja = Abeja('insecto', 1)

mi_perro.hablar()
mi_vaca.hablar()
# Guau!
# Muuu!

mi_vaca.describeme()
mi_abeja.describeme()
# Soy un Animal del tipo Vaca
# Soy un Animal del tipo Abeja

mi_abeja.picar()
# Picar!
```

super()

¿Para qué sirve?



La función **super** nos permite acceder a los métodos de la clase padre desde una de sus hijas.

```
class Animal:
    def __init__(self, especie, edad):
        self.especie = especie
        self.edad = edad
    def hablar(self):
        pass

    def moverse(self):
        pass

    def describeme(self):
        print("Soy un Animal del tipo", type(self).__name__)
```

Volvamos al ejemplo de Animal y Perro...

¿Para qué sirve?



```
class Perro(Animal):
    def __init__(self, especie, edad, dueño):
        # Alternativa 1
        # self.especie = especie
        # self.edad = edad
        # self.dueño = dueño

        # Alternativa 2
        super().__init__(especie, edad)
        self.dueño = dueño
```

```
mi_perro = Perro('mamífero', 7, 'Luis')
mi_perro.especie
mi_perro.edad
mi_perro.dueño
```

Tal vez queramos que nuestro Perro tenga un parámetro extra en el constructor, como podría ser el **dueño**. Para realizar esto tenemos dos alternativas:

¿Para qué sirve?



```
class Perro(Animal):
    def __init__(self, especie, edad, dueño):
        # Alternativa 1
        # self.especie = especie
        # self.edad = edad
        # self.dueño = dueño

        # Alternativa 2
        super().__init__(especie, edad)
        self.dueño = dueño
```

```
mi_perro = Perro('mamífero', 7, 'Luis')
mi_perro.especie
mi_perro.edad
mi_perro.dueño
```

1. Podemos crear un nuevo **`__init__`** y guardar todas las variables una a una.
1. O podemos usar **`super()`** para llamar al **`__init__`** de la clase padre que ya aceptaba la especie y edad y sólo asignar la variable nueva manualmente.

Herencia múltiple

Herencia múltiple

Hemos visto cómo se podía crear una clase padre que heredaba de una clase hija, pudiendo hacer uso de sus métodos y atributos.

La herencia múltiple es similar, pero una clase hereda de varias clases padre en vez de una sola.



Para pensar

¿Qué diferencias se pueden detectar?

```
class Clase1:  
    pass  
class Clase2:  
    pass  
class Clase3(Clase1, Clase2):  
    pass
```

```
class Clase1:  
    pass  
class Clase2(Clase1):  
    pass  
class Clase3(Clase2):  
    pass
```

Herencia múltiple

En Python es posible realizar herencia múltiple.

Anteriormente hemos visto cómo se podía crear una clase padre que heredaba de una clase hija, pudiendo hacer uso de sus métodos y atributos.

La herencia múltiple es similar, pero una clase hereda de varias clases padre en lugar de una sola.

Ejemplo



Por un lado tenemos dos clases Clase1 y Clase2, y por otro tenemos la **Clase3 que hereda de las dos anteriores**. Por lo tanto, heredará todos los métodos y atributos de ambas.

```
class Clase1:  
    pass  
class Clase2:  
    pass  
class Clase3(Clase1, Clase2):  
    pass
```


Ejemplo



```
class Clase1:  
    pass  
class Clase2(Clase1):  
    pass  
class Clase3(Clase2):  
    pass
```

Es posible también que una clase herede de otra clase y a su vez otra clase herede de la anterior.



PARA RECORDAR

Hasta ahora sabíamos que las clases hijas heredan los métodos de las clases padre, pero también pueden reimplementarlos de manera distinta. Este punto nos plantea el siguiente interrogante:

Si llamo a un método que todas las clases tienen en común ¿A cuál se llama?

Pues bien, existe una forma de saberlo. 🙌

Method Order Resolution

Sabremos a qué método se llama, consultando al MRO. **Esta función nos devuelve una tupla con el orden de búsqueda de los métodos.**

Como era de esperar se empieza en la propia clase y se va subiendo hasta la clase padre, de izquierda a derecha.

Method Order Resolution



Una curiosidad es que al final del todo vemos la clase `object`. Aunque pueda parecer raro, es correcto ya que en realidad todas las clases en Python heredan de una clase genérica `object`, aunque no lo especifiquemos explícitamente.

```
class Clase1:
    pass
class Clase2:
    pass
class Clase3(Clase1, Clase2):
    pass

print(Clase3.__mro__)
# (<class '__main__.Clase3'>, <class '__main__.Clase1'>, <class '__main__.Clase2'>, <class 'object'>)
```

Method Order Resolution



También podemos tener una clase heredando de otras tres. Vemos que el MRO depende del orden en el que las clases son pasadas: 1, 3, 2.

```
class Clase1:
    pass
class Clase2:
    pass
class Clase3:
    pass
class Clase4(Clase1, Clase3, Clase2):
    pass
print(Clase4.__mro__)
# (<class '__main__.Clase4'>, <class '__main__.Clase1'>, <class '__main__.Clase3'>, <class '__main__.Clase2'>, <class 'object'>)
```

Polimorfismo

¿De qué se trata?

El término aplicado a la programación hace referencia a que los objetos pueden tomar diferentes formas. ¿Pero qué significa esto?

Significa que, **objetos de diferentes clases, pueden ser accedidos utilizando el mismo interfaz, mostrando un comportamiento distinto (tomando diferentes formas) según cómo sean accedidos.**

¿Para qué se utiliza?

Si la técnica de polimorfismo, siendo una propiedad de la POO (Programación Orientada a Objetos), implica la capacidad de tomar más de una forma, una operación puede presentar diferentes comportamientos en diferentes instancias.

El comportamiento depende de los tipos de datos utilizados en la operación. El polimorfismo es ampliamente utilizado en la aplicación de la herencia.

Veamos cómo funciona 🙌

¿Cómo se utiliza?



```
✓ [15] class Persona():
      def __init__(self):
          self.cedula = 13765890
      def mensaje(self):
          print("mensaje desde la clase Persona")

      class Obrero(Persona):
          def __init__(self):
              self.__especialista = 1
          def mensaje(self): #Aquí tenemos al método Polimórfico
              print("mensaje desde la clase Obrero")

✓ [16] persona1 = Persona()

      persona1.mensaje()

      mensaje desde la clase Persona

✓ [17] obrero1 = Obrero()

      obrero1.mensaje()

      mensaje desde la clase Obrero
```

Podemos sustituir un método proveniente de la Clase Padre, en la Clase Hija. Se debe definir un método con el mismo nombre y parámetros, pero debe tomar otra conducta.

Es básicamente lo que veníamos haciendo sin saber que se llamaba Polimorfismo. 🤖

Duck typing

¿Qué es?

El término polimorfismo visto desde el punto de vista de Python es complicado de explicar sin hablar del duck typing. Es un concepto que aplica a ciertos lenguajes orientados a objetos y que tiene origen en la siguiente frase:

If it walks like a duck and it quacks like a duck, then it must be a duck

(Si camina como un pato y habla como un pato, entonces tiene que ser un pato)





PARA RECORDAR

¿Y qué relación tienen los patos con la programación? Pues bien, se trata de un símil en el que los patos son objetos y hablar/andar métodos. Es decir, que **si un determinado objeto tiene los métodos que nos interesan, nos basta, siendo su tipo irrelevante.**



¿Cómo funciona?



Una vez entendido el origen del concepto, veamos lo que realmente significa esto en Python. En pocas palabras, **a Python le dan igual los tipos de los objetos, lo único que le importan son los métodos.**

```
class Pato:  
    def hablar(self):  
        print("¡Cua!, Cua!")
```

Definamos una clase **pato** con un método **hablar ()**.

Y llamamos al método de la siguiente forma:

```
p = Pato()  
p.hablar()  
# ¡Cua!, Cua!
```



Método hablar ()



En Python **no es necesario especificar los tipos**, simplemente decimos que el parámetro de entrada tiene el nombre **x**.

Hasta aquí nada nuevo, pero vamos a definir una función **llama_hablar ()**, que llama al método **hablar ()** del objeto que se le pase.

```
def llama_hablar(x):  
    x.hablar()
```



Método hablar ()



Cuando Python entra en la función y evalúa `x.hablar()`, le da igual el tipo al que pertenezca `x` siempre y cuando tenga el método `hablar()`.

```
def llama_hablar(x):  
    x.hablar()
```

Esto es el duck typing en todo su esplendor. 🦆



Método hablar ()



Definamos tres clases de animales distintas que implementan el método **hablar()**.

```
class Perro:
    def hablar(self):
        print("¡Guau, Guau!")

class Gato:
    def hablar(self):
        print("¡Miau, Miau!")

class Vaca:
    def hablar(self):
        print("¡Muuu, Muuu!")
```

Nótese que no existe **herencia** entre ellas, son clases totalmente independientes. De haberla estaríamos hablando de **polimorfismo**.



Método hablar ()



```
llama_hablar(Perro())  
llama_hablar(Gato())  
llama_hablar(Vaca())
```

```
# ¡Guau, Guau!  
# ¡Miau, Miau!  
# ¡Muuu, Muuu!
```

Y, como es de esperar, la función **llama_hablar()** funciona correctamente con todos los objetos.

Otra forma de verlo, es iterando una lista con diferentes animales, donde animal toma los valores de cada objeto animal.

```
lista = [Perro(), Gato(), Vaca()]  
for animal in lista:  
    animal.hablar()
```

```
# ¡Guau, Guau!  
# ¡Miau, Miau!  
# ¡Muuu, Muuu!
```

Conclusiones

- ✓ Python es un lenguaje que soporta el duck typing, lo que hace que el tipo de los objetos no sea tan relevante, siendo más importante lo que sus métodos pueden hacer.
- ✓ Otros lenguajes como Java, no soportan el duck typing, pero se puede conseguir un comportamiento similar cuando los objetos comparten un interfaz (si existe herencia entre ellos). Este concepto relacionado es el **polimorfismo**.
- ✓ El duck typing está en todos lados, desde la función **len()** hasta el uso del operador *****



Volviendo al Poliformismo



Al ser un lenguaje con tipado dinámico y permitir duck typing, en Python no es necesario que los objetos compartan un interfaz, simplemente basta con que tengan los métodos que se quieren llamar.

Supongamos que tenemos una clase **Animal** con un método **hablar()**.

```
class Animal:
    def hablar(self):
        pass
```

hablar () y Poliformismo



Por otro lado tenemos otras dos clases, **Perro**, **Gato** que heredan de la anterior. Además, implementan el método **hablar()** de una forma distinta.

```
class Perro(Animal):  
    def hablar(self):  
        print("Guau!")  
  
class Gato(Animal):  
    def hablar(self):  
        print("Miau!")
```

hablar () y Poliformismo



A continuación creamos un objeto de cada clase y llamamos al método **hablar()**. Podemos observar que cada animal se comporta de manera distinta al usar **hablar()**.

```
for animal in Perro(), Gato():  
    animal.hablar()  
  
# Guau!  
# Miau!
```



PARA RECORDAR

En el caso anterior, la variable **animal** ha ido “tomando las formas” de **Perro** y **Gato**. Sin embargo, nótese que al tener tipado dinámico este ejemplo hubiera funcionado igual sin que existiera herencia entre **Perro** y **Gato**, como sucede en duck typing.

Herencia y Encapsulamiento

¡Por último!

A través de la aplicación del concepto de Encapsulamiento, podemos indicarle a Python por ejemplo, que no queremos que un método sea heredado por una clase hija.

Veamos a continuación el notebook: [*Herencia y Encapsulamiento.ipynb*](#)



Herencia múltiple

Crear una herencia múltiple, trabajando con Mamífero, Cetáceo, AnimalMarino.

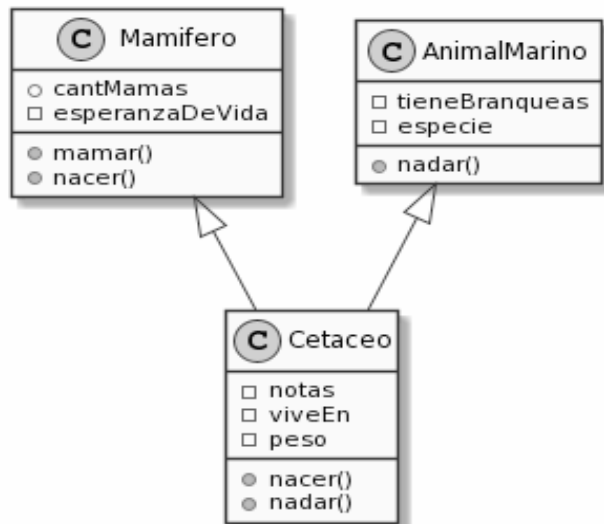
Duración: 20 minutos



ACTIVIDAD EN CLASE

Herencia múltiple

Realizar la herencia múltiple respetando este diagrama de Clases UML. Y crear dos Cetáceos.



Especie	Longitud (m)	Peso (kg)	Notas
Cochito o Vaquita marina	1,2-1,5	30-55	Cetáceos más pequeño. en Peligro crítico de extinción.
Delfín común	2,4	70-110	
Delfín mular	3,7	150-650	
Narval	5	800-1600	
Ballena franca pigmea	6,5	3000-3500	Misticetos más pequeños
Orca	9,7	2600-9000	Delfínidos más grandes
Yubarta	13,7-15,2	25 000-40 000	
Cachalote	14,9-20	13 000-14 000	Odontoceto más grandes
Ballena azul	30	110 000	Animal más grande de todos



MATERIAL AMPLIADO

Recursos multimedia

Ejercicios

- ✓ [Ejercicios de Repaso](#)
- ✓ [Ejercicios Clase En Vivo](#)

¿Preguntas?